SouverainRecrut

Industrialisation, sécurisation et déploiement d'une API backend

Compte rendu du TP CI/CD

Colleen Pezon
Mastère Expert en Développement et Déploiement de Solutions Numériques Sécurisées · RNCP 40166
Module CI/CD (formateur : Alimou Diallo) · Sup de Vinci Rennes · novembre 2026

Dépôt GitHub (intégration continue, branche Dev) : https://github.com/pcolleen/souverainrecrut/tree/Dev
Dépôt GitLab (déploiement) : https://gitlab.com/pcolleen/souverainrecrut-deploiement

Sommaire

Introduction

1. Intégration continue avec Jenkins

Étapes 1 à 7 : outillage, tests, SonarQube, Jenkins, Jenkinsfile, image Docker, Docker Hub

2. Provisionnement et déploiement

Étapes 8 à 12 : serveur, Terraform, Ansible, Docker Compose, GitLab CI/CD

3. Vérification et résilience

Étapes 13 à 15 : API en production, persistance, rollback et restauration

Bilan

Introduction

L'objectif du TP est de construire une chaîne de livraison complète pour l'API backend de SouverainRecrut.

J'ai organisé le travail en deux chaînes :

- **Intégration continue (Jenkins) :** GitHub (Dev) → tests → SonarQube → build Docker → Trivy → Docker Hub.
- **Déploiement (GitLab CI/CD) :** serveur Ubuntu → Terraform → Ansible → pull de l'image Docker Hub → Docker Compose + PostgreSQL.

Le point de jonction est l'image Docker publiée : **Jenkins la construit, GitLab la déploie sans jamais la reconstruire.** La version déployée est donc exactement celle qui a été testée et scannée.

1. Intégration continue avec Jenkins

Étape 1 : Outillage CI en conteneurs

> **Ce que j'ai mis en place :** un fichier `docker-compose.tools.yml` qui démarre Jenkins, SonarQube (avec leurs bases) et une base PostgreSQL jetable pour les tests. Jenkins utilise une image personnalisée (`Dockerfile.jenkins`) qui embarque Node 22, le CLI Docker, sonar-scanner et les plugins nécessaires.

> **Pourquoi :** n'avoir que Docker et Git à installer sur la machine, et pouvoir recréer tout l'outillage à l'identique en une commande.

> **Résultat :** Jenkins accessible sur le port 8080, SonarQube sur le port 9000. Les secrets du compose sont dans un `.env` non versionné (modèle `.env.example`).

Étape 2 : Tests automatisés du backend

> **Ce que j'ai mis en place :** Le dépôt ne contenait que des scripts de test vides : j'ai configuré **Jest + Supertest** et écrit des tests qui appellent l'API sur une vraie base PostgreSQL.

> **Pourquoi :** tester l'API dans des conditions proches de la production plutôt qu'avec des mocks.

> **Résultat :** Les tests tournent dans la pipeline et produisent un rapport de couverture `coverage/lcov.info`, réutilisé par SonarQube.

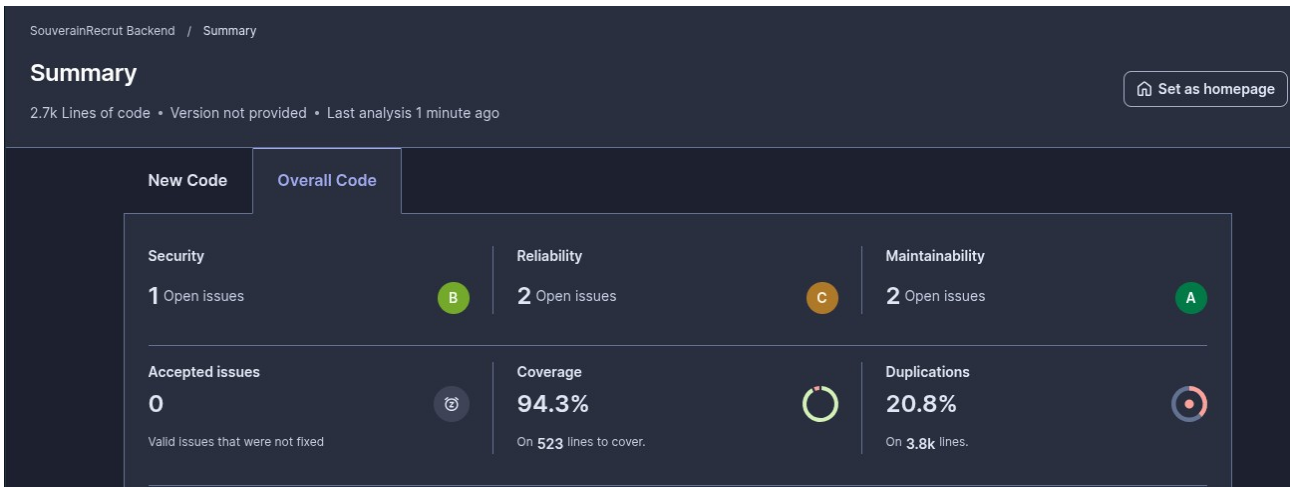


Figure 1 : SonarQube, première analyse sur l'ensemble du code (avant mes tests backend) : couverture 94,3 %, 20,8 % de duplications, 5 points ouverts.

Étape 3 : Analyse qualité avec SonarQube

> **Ce que j'ai mis en place :** Création du projet `souverainrecrut-backend` génération d'un token, et fichier `sonar/sonar-project.properties` qui indique les sources, les exclusions et le rapport de couverture.

> **Pourquoi :** mesurer bugs, vulnérabilités, code smells, duplications et couverture, et bloquer la livraison si le Quality Gate échoue.,

Résultat : Quality Gate au statut PASSED.

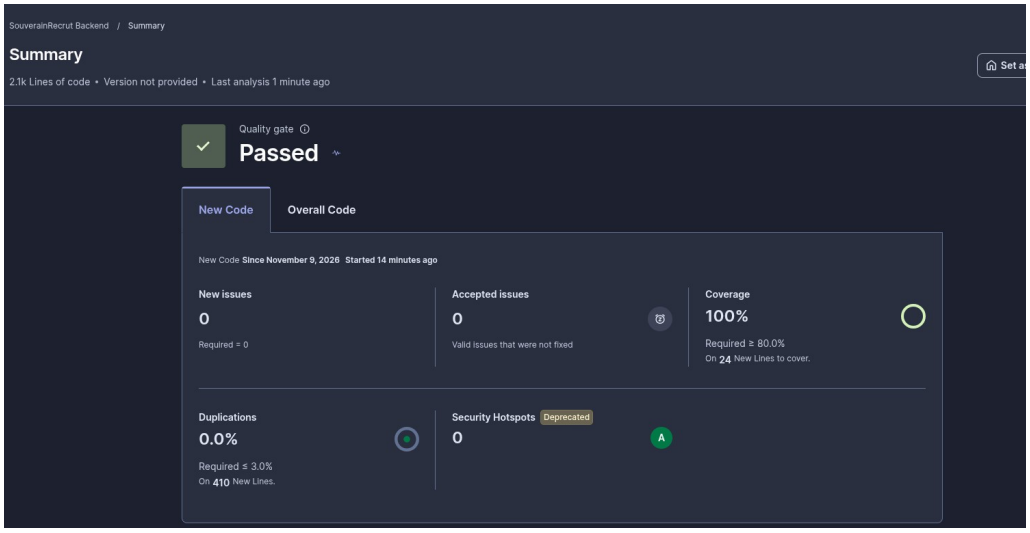


Figure 2 : SonarQube : Quality Gate PASSED sur le nouveau code (0 nouveau problème, couverture 100 %, 0 % de duplication).

Étape 4 : Configuration de Jenkins

Ce que j'ai mis en place : Ajout de deux credentials de type Secret text (`sonar-token`, `docker_hub`), déclaration du serveur SonarQube (`http://sonarqube:9000`, nom du service sur le réseau Docker), puis création d'un job Pipeline « from SCM » sur la branche `Dev`.

Pourquoi : aucun secret n'est écrit dans le dépôt : Jenkins les injecte au moment du build et les masque dans les logs.

Résultat : Jenkins scrute GitHub toutes les 5 minutes et lance la pipeline à chaque nouveau commit. J'ai choisi la scrutation car un Jenkins local n'est pas joignable par un webhook GitHub.

Étape 5 : Le Jenkinsfile

Ce que j'ai mis en place : Un `Jenkinsfile` versionné dans `integration-continue/` qui enchaîne les étapes ci-dessous. Chaque étape est un point de contrôle : si elle échoue, la suite n'est pas exécutée.

Étape	Ce qu'elle fait	Bloque si...
Checkout, install, Prisma	npm ci, prisma generate	échec
Tests	Jest + Supertest sur PostgreSQL, rapport de couverture	un test échoue
SonarQube	analyse + attente du Quality Gate	Quality Gate rouge
Image Docker	build multi-stage, tag <version>-<sha court>	échec du build
Scan Trivy	rapport JSON archivé, HIGH/CRITICAL affichés	faille CRITICAL corrigeable
Publication Docker Hub	docker push	contrôles précédents en échec

Étape 6 : Image Docker du backend

Ce que j'ai mis en place : J'ai repris le Dockerfile fourni en **multi-stage** (compilation séparée de l'image finale), avec un `.dockerignore`, un utilisateur non-root, le port 4500 et un HEALTHCHECK sur `/v1/health`.

Pourquoi : obtenir une image légère, reproductible et sans secret : DATABASE_URL et JWT_SECRET ne sont fournis qu'à l'exécution.

Résultat : Chaque image est taguée `<version>-<sha du commit>` (ex. `1.0.0-387bccc`) : on sait toujours quel code tourne en production.

Étape 7 : Publication sur Docker Hub

> **Ce que j'ai mis en place :** Création d'un access token Docker Hub (Read & Write), stocké dans Jenkins et transmis via `docker login --password-stdin`. La publication n'a lieu que si toutes les étapes précédentes sont vertes.

> **Pourquoi :** ne publier que des images testées, analysées et scannées, sans jamais exposer le token.

> **Résultat :** Build n°7 entièrement vert, image disponible sur `colleenp/souverainrecrut-backend`.

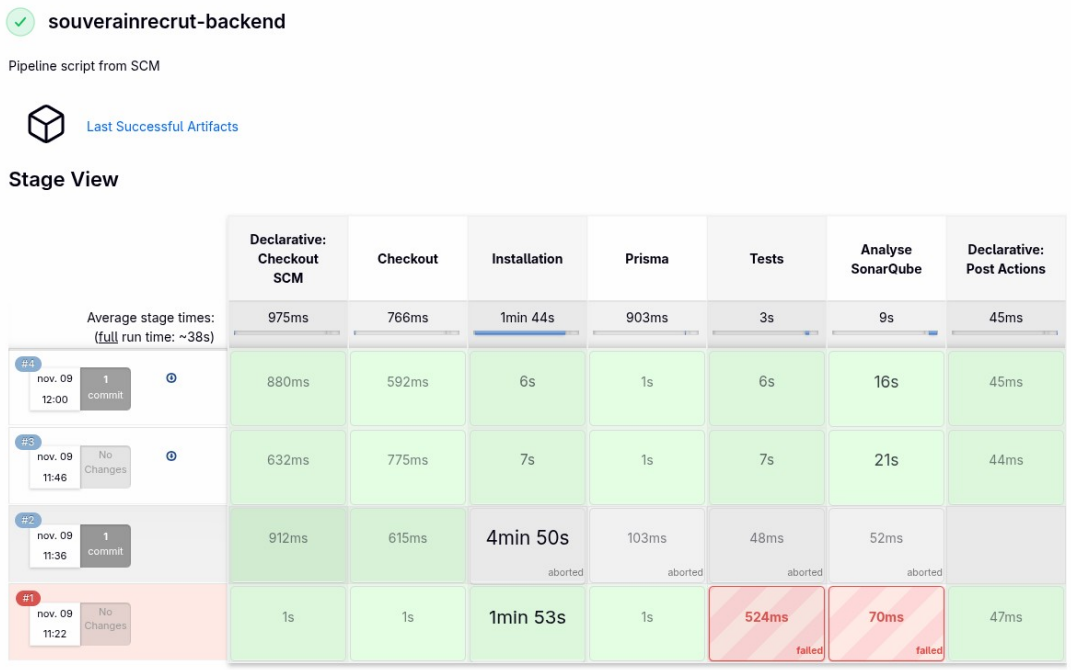


Figure 3 : Jenkins : Stage View des builds #1 à #4. Le build #1 échoue aux tests et à l'analyse, les builds #3 et #4 passent toutes les étapes.

2. Provisionnement et déploiement

Étape 8 : Choix du serveur

> **Ce que j'ai mis en place :** j'ai utilisé un serveur réel, un Scaleway Dedibox VPS sous Ubuntu 24.04, à la place d'une VM VMware locale.

> **Pourquoi :** déployer dans des conditions réelles : API accessible depuis Internet, vraies contraintes de sécurité (SSH, pare-feu).

> **Résultat :** serveur joignable en SSH, avec accès Internet.

Étape 9 : Terraform

> **Ce que j'ai mis en place :** Une configuration Terraform (`providers.tf`, `variables.tf`, `main.tf`, `outputs.tf`) qui décrit le serveur (IP, utilisateur, clé, port) et génère automatiquement l'inventaire Ansible. Des validations refusent une IP invalide, l'utilisateur root ou un port privilégié.

> **Pourquoi :** la gamme Dedibox n'a pas de provider Terraform (comme VMware Workstation sans vCenter, cas prévu par le cahier des charges). Plutôt que de simuler une ressource, j'ai utilisé Terraform sur ce qui est réellement gérable : une source de vérité unique pour l'infrastructure. Avec un fournisseur cloud, il suffirait d'ajouter le provider et la ressource serveur.

> **Résultat :** init, validate et plan passent ; l'inventaire `ansible/inventory.ini` est généré, ainsi que les sorties utiles (commande SSH, URL de l'API, valeurs à reporter dans GitLab).

```
Terraform : description du serveur et inventaire Ansible genere

$ terraform validate
Success! The configuration is valid.


$ terraform plan
No changes. Your infrastructure matches the configuration.

$ terraform output
ansible_inventory = "./../ansible/inventory.ini"
api_url = "http://185.186.78.239:4500/v1"
gitlab_variables_non_secretes = {
  "API_PORT" = "4500"
  "DEPLOY_PATH" = "/opt/souverainrecrut"
  "SSH_HOST" = "185.186.78.239"
  "SSH_USER" = "deploy"
}
ssh_command = "ssh -i ~/.ssh/id_ed25519 deploy@185.186.78.239"
swagger_url = "http://185.186.78.239:4500/api-docs"

$ cat ../ansible/inventory.ini
# Fichier GÉNÉRÉ par Terraform (deploiement/terraform) : ne pas modifier à la main.
[deploy]
serveur ansible_host=185.186.78.239 ansible_user=deploy ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o IdentitiesOnly=yes'
```

Figure 4 : Terraform : init, validate, plan et génération de l'inventaire Ansible.

Étape 10 : Configuration du serveur avec Ansible

Ce que j'ai mis en place : Deux playbooks : `bootstrap.yml` (exécuté une fois en root : crée l'utilisateur `deploy` et autorise la clé de la pipeline) puis `playbook.yml` (mises à jour, Docker + Compose, pare-feu UFW sur les ports 22 et 4500, fail2ban, SSH durci).

Pourquoi : préparer le serveur de façon automatique et rejouable, et le sécuriser : connexion par clé uniquement, root et mots de passe refusés. J'ai ajouté des garde-fous pour ne pas me verrouiller hors du serveur : validation de chaque clé SSH, test d'une nouvelle connexion avant de durcir SSH, annulation automatique si ce test échoue.

Résultat : le playbook est idempotent (second passage : changed=0) et le serveur est conforme.

```
Ansible : bootstrap, playbook, redemarrage, idempotence

$ ansible-playbook bootstrap.yml -e ansible_user=root      # une seule fois
serveur                    : ok=12   changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

$ ansible-playbook playbook.yml                            # 1er passage
serveur                    : ok=26   changed=2    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
TASK [Tester une NOUVELLE connexion par clé (session indépendante de celle en cours)] ***
   -> garde-fou : nouvelle connexion par cle testee avant de conclure
    "msg": "All assertions passed"

$ ansible deploy -b -m reboot                              # nouveau noyau
   noyau actif apres redemarrage : 6.8.0-149-generic

$ ansible-playbook playbook.yml                            # 2e passage : idempotence
serveur                    : ok=25   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
   -> changed=0 : le playbook est rejouable sans effet
```

Figure 5 : Ansible : bootstrap, playbook, redémarrage puis second passage idempotent (changed=0).

```
Garde-fou Ansible : authorized_keys (valide, idempotent, refuse une ligne corrompue)

# Garde-fou authorized_keys : preuve en local (fichier dont la derniere ligne n a pas de retour a la ligne)

$ echo "$CLE_CI" >> authorized_keys        # ANCIENNE methode
lignes: 1 | deux cles collees sur une meme ligne: 1 (ERREUR : serveur verrouille)

$ ansible-playbook bootstrap.yml           # NOUVELLE methode (lineinfile + validation stricte)
TASK [ansible.builtin.lineinfile] ******************************************
changed: [localhost]
PLAY RECAP *********************************************************************
lignes: 2 | cles valides lues par ssh-keygen: 2 | cles collees: 0

$ ansible-playbook t2.yml                  # idempotence + ligne corrompue
TASK [2e passage (idempotence)] ********************************************
ok: [localhost]
TASK [Ligne CORROMPUE (doit etre refusee)] *********************************
fatal: [localhost]: FAILED! => {"changed": false, "msg": "failed to validate: rc:1 error:")
...ignoring
TASK [Verdict] *************************************************************
ok: [localhost] => {
    "msg": "corruption refusee: True"
fichier inchange apres le refus: OUI
```

Figure 6 : Garde-fou : l'ancienne méthode collait deux clés sur une ligne ; la nouvelle valide chaque clé et refuse une ligne corrompue.

```
Serveur Scaleway : acces SSH par cle uniquement, pare-feu, Docker

$ ssh -i ~/.ssh/id_ed25519 deploy@185.186.78.239 'id; echo connexion par cle: OK'
uid=1000(deploy) gid=1000(deploy) groups=1000(deploy),988(docker)
connexion par cle: OK

$ ssh -i ~/.ssh/id_ed25519 root@185.186.78.239 true; echo "code=$? -> attendu: refus"
root@185.186.78.239: Permission denied (publickey).
code=255 -> attendu: refus

$ ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password -o BatchMode=yes deploy@185.186.78.239 true;
echo "code=$? -> attendu: refus"
deploy@185.186.78.239: Permission denied (publickey).
code=255 -> attendu: refus

$ ssh -i ~/.ssh/id_ed25519 deploy@185.186.78.239 "sudo sshd -T | grep -E
'^(passwordauthentication|permitrootlogin|kbdinteractiveauthentication) '"
permitrootlogin no
passwordauthentication no
kbdinteractiveauthentication no

$ ssh -i ~/.ssh/id_ed25519 deploy@185.186.78.239 "sudo ufw status verbose | sed -n '1,11p'"
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
4500/tcp                   ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
4500/tcp (v6)              ALLOW IN    Anywhere (v6)

$ ssh -i ~/.ssh/id_ed25519 deploy@185.186.78.239 "systemctl is-active docker fail2ban ufw | paste -sd' ' ; docker
--version; docker compose version; uname -r"
active active active
Docker version 29.9.3, build 5457199
Docker Compose version v5.7.0
6.8.0-149-generic
```

Figure 7 : Serveur : SSH par clé uniquement, pare-feu UFW (22 et 4500), Docker et fail2ban actifs.

Étape 11 : Docker Compose et script de déploiement

Ce que j'ai mis en place : Un `deployment/docker-compose.yml` avec trois services : PostgreSQL (volume persistant), un conteneur dédié aux migrations Prisma, puis l'API (healthchecks, politiques de redémarrage, réseau dédié). Un script `deploy.sh` déploie un tag donné : sauvegarde de la base, pull de l'image, migrations, démarrage, contrôle de santé.

Pourquoi : les migrations s'exécutent une seule fois, avant l'API, dans un conteneur séparé : c'est la stratégie maîtrisée demandée. La sauvegarde avant chaque déploiement permet de revenir en arrière côté données.

Résultat : premier déploiement réussi : image `1.0.0-387bccc` tirée de Docker Hub, migrations appliquées, API saine.

```
Deploiement sur le serveur : image tiree de Docker Hub, migrations, sante

$ ./deploy.sh 1.0.0-387bccc
[deploy] Version demandée : 1.0.0-387bccc (version actuelle : 1.0.0-0000000)
 Image postgres:16-alpine Pulled
 Image colleenp/souverainrecrut-backend:1.0.0-387bccc Pulled
[deploy] Aucune base en cours d'exécution : première installation, pas de sauvegarde.
[deploy] Démarrage de 1.0.0-387bccc (migrations Prisma, puis API)...
 Container souverainrecrut-postgres-1 Healthy
 Container souverainrecrut-migrate-1 Exited
NAME                          IMAGE                                          COMMAND                  SERVICE
CREATED          S
souverainrecrut-backend-1     colleenp/souverainrecrut-backend:1.0.0-387bccc   "docker-entrypoint.s…"   backend    14
seconds ago    U
souverainrecrut-postgres-1    postgres:16-alpine                             "docker-entrypoint.s…"   postgres   15
seconds ago    U
[deploy] Déploiement réussi : 1.0.0-387bccc
code de sortie: 0
```

Figure 8 : Premier déploiement : image tirée de Docker Hub, migrations Prisma, API en bonne santé.

Étape 12 : Pipeline GitLab CI/CD

Ce que j'ai mis en place : Un `.gitlab-ci.yml` en trois étapes : `validate:tag` (vérifie que le tag existe sur Docker Hub), `deploy:production` (approbation manuelle, connexion SSH, envoi des fichiers et lancement de `deploy.sh`), `verify:production` (contrôle des conteneurs et de l'API). Les secrets (token Docker Hub, clé SSH…) sont dans les variables masquées et protégées de GitLab.

Pourquoi : garder une validation humaine avant la production, et ne jamais reconstruire l'image : on déploie exactement celle validée par Jenkins.

Résultat : pipeline vert ; sur le serveur, une nouvelle ligne apparaît dans `releases.log`, le `.env` est régénéré et une sauvegarde est faite.

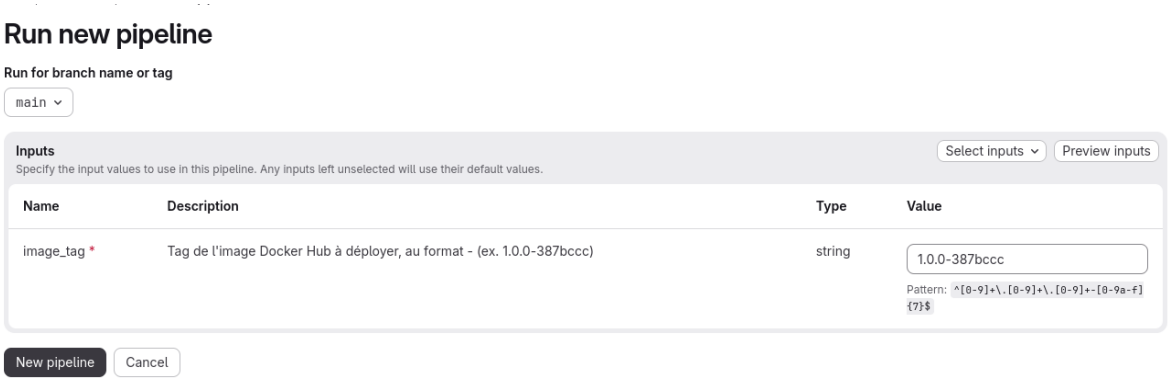


Figure 9 : GitLab : lancement du pipeline avec le tag de l'image à déployer (format contrôlé par une expression régulière).

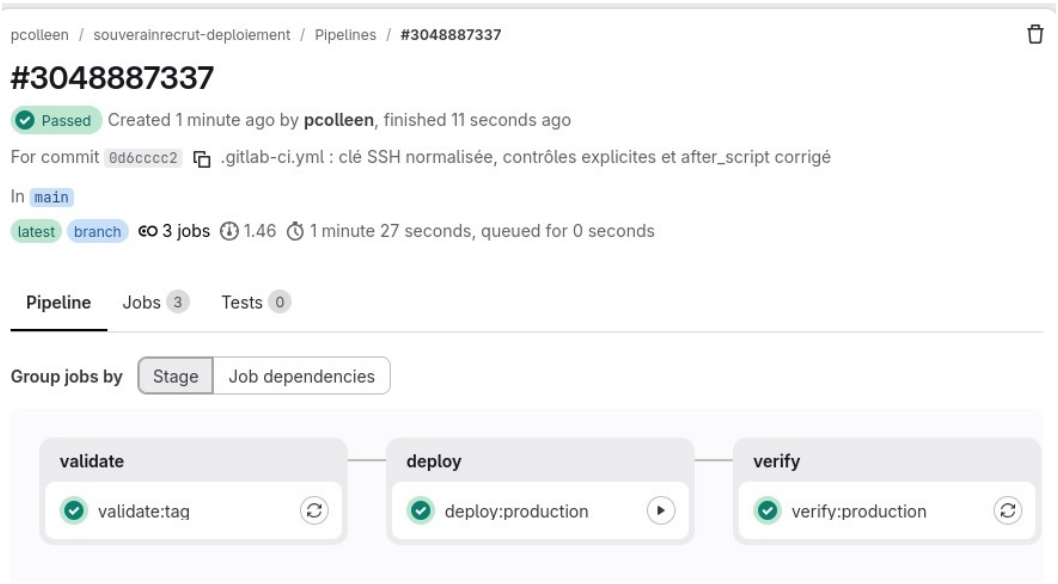


Figure 10 : GitLab : pipeline réussi, avec les trois jobs validate:tag, deploy:production (déclenché manuellement) et verify:production.

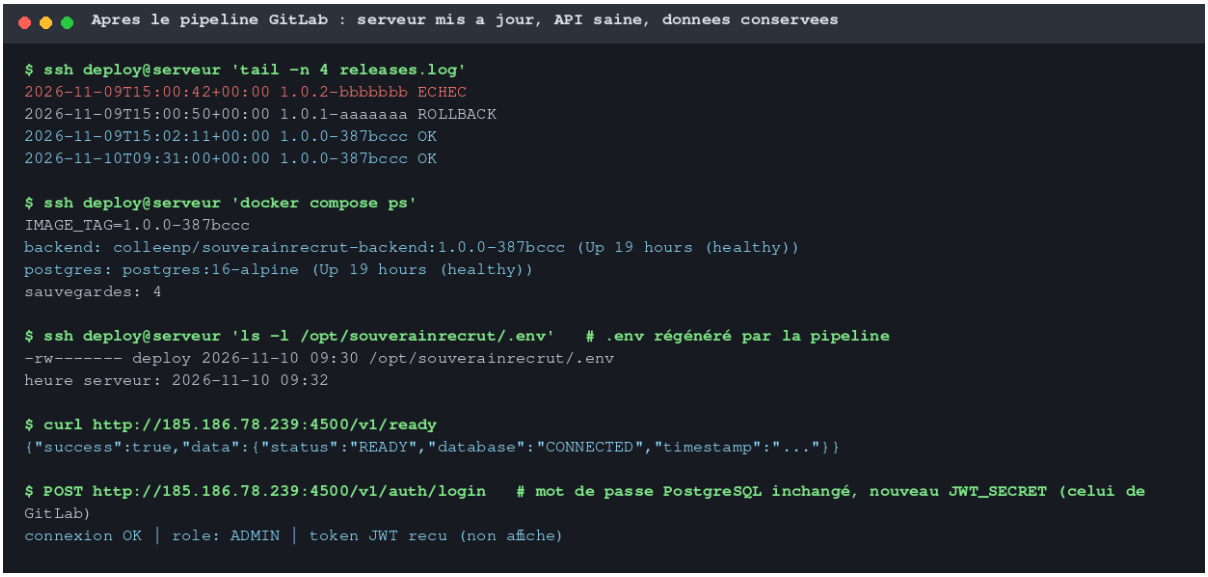


Figure 11 : Après le pipeline GitLab : releases.log mis à jour, .env régénéré, sauvegarde faite, API saine.

3. Vérification et résilience

Étape 13 : Vérification de l'API en production

Ce que j'ai mis en place : une série d'appels depuis Internet sur les principaux endpoints.

Résultat : API en ligne (`/v1/health`), base connectée (`/v1/ready`), route protégée refusée sans token (401), connexion admin avec JWT, création d'un candidat en base. La documentation Swagger est servie sur `/api-docs`.

```
API deployee sur le VPS (Internet) : sante, authentification, routes protegees, ecriture

$ curl http://185.186.78.239:4500/v1/health
{"success":true,"data":{"status":"UP","timestamp":"2026-11-
09T14:57:33.877Z","uptime":31.843889345,"environment":"production"}}

$ curl http://185.186.78.239:4500/v1/ready
{"success":true,"data":{"status":"READY","database":"CONNECTED","timestamp":"2026-11-09T14:57:34.172Z"}}

$ curl -o /dev/null -w '%{http_code}' http://185.186.78.239:4500/v1/jobs   # sans token
401

$ creation de l'administrateur (dans le conteneur, secrets par stdin)
administrateur pret

$ POST http://185.186.78.239:4500/v1/auth/login   # identifiants de l'administrateur (non affichés)
connexion OK | role: ADMIN | token JWT recu ( 192 caracteres, non affiche)

$ GET http://185.186.78.239:4500/v1/auth/me   # avec token
{'email': 'admin@souverainrecrut.local', 'role': 'ADMIN'}

$ POST http://185.186.78.239:4500/v1/candidates   # ecriture en base
cree: Candidate Demo

$ GET http://185.186.78.239:4500/v1/dashboard/stats
{"success":true,"data":{"users":1,"candidates":1,"jobs":0,"applications":0,"pendingApplications":0}}
```

Figure 12 : API vue depuis Internet : santé, base connectée, route protégée (401), connexion, écriture en base.

Étape 14 : Persistance des données

Ce que j'ai mis en place : destruction de tous les conteneurs (`docker compose down`) puis redéploiement.

Résultat : les données sont toujours présentes après redéploiement : elles vivent dans le volume PostgreSQL, pas dans les conteneurs.

```
Serveur : persistance des donnees apres destruction des conteneurs

$ curl http://185.186.78.239:4500/v1/dashboard/stats        # AVANT
{"success":true,"data":{"users":1,"candidates":1,"jobs":0,"applications":0,"pendingApplications":0}}

$ docker compose down               # conteneurs détruits (le volume pgdata est conservé)
Container souverainrecrut-backend-1 Removed
Container souverainrecrut-backend-1 Stopped
Container souverainrecrut-migrate-1 Removed
Container souverainrecrut-migrate-1 Stopped
Container souverainrecrut-postgres-1 Removed
Container souverainrecrut-postgres-1 Stopped
Network souverainrecrut_backend Removed
volume conservé: souverainrecrut_pgdata
API pendant l'arrêt : 000 (000 = injoignable, attendu)

$ ./deploy.sh 1.0.0-387bccc          # redéploiement
[deploy] Version demandée : 1.0.0-387bccc (version actuelle : 1.0.0-387bccc)
[deploy] Aucune base en cours d'exécution : première installation, pas de sauvegarde.
[deploy] Démarrage de 1.0.0-387bccc (migrations Prisma, puis API)...
[deploy] Déploiement réussi : 1.0.0-387bccc

$ curl http://185.186.78.239:4500/v1/dashboard/stats        # APRÈS : les données ont survécu
{"success":true,"data":{"users":1,"candidates":1,"jobs":0,"applications":0,"pendingApplications":0}}
```

Figure 13 : Persistance : conteneurs détruits puis redéployés, les données sont conservées.

Étape 15 : Rollback automatique et restauration de la base

Ce que j'ai mis en place : Dans `deploy.sh`, un retour arrière automatique : si la nouvelle version n'est pas saine en 45 s, la précédente est remise en service. Un script `restore-db.sh` restaure la base depuis une sauvegarde. Pour le tester, j'ai créé sur le serveur deux versions de test non publiées : une saine et une volontairement cassée.

Pourquoi : le rollback de l'image ne suffit pas si une migration a modifié le schéma : d'où la sauvegarde systématique avant chaque déploiement et la procédure de restauration documentée.

> **Résultat :** la version cassée est détectée en 45 s et la version précédente reprend automatiquement (ligne ECHEC puis ROLLBACK dans releases.log). Après une perte de données simulée, la base est restaurée depuis la sauvegarde d'avant déploiement.

```
Serveur : mise a jour, version cassee detectee, retour arriere automatique

# Versions de test creees sur le serveur (jamais publiees) : 1.0.1 = copie saine, 1.0.2 = cassee (CMD qui plante)
colleenp/souverainrecrut-backend:1.0.0-387bccc
colleenp/souverainrecrut-backend:1.0.1-aaaaaaa
colleenp/souverainrecrut-backend:1.0.2-bbbbbbb

$ ./deploy.sh 1.0.1-aaaaaaa            # mise a jour saine
[deploy] Version demandée : 1.0.1-aaaaaaa (version actuelle : 1.0.0-387bccc)
[deploy] Base sauvegardée : backups/avant-1.0.1-aaaaaaa-20261109-145922.sql.gz
[deploy] Démarrage de 1.0.1-aaaaaaa (migrations Prisma, puis API)...
[deploy] Déploiement réussi : 1.0.1-aaaaaaa

$ ./deploy.sh 1.0.2-bbbbbbb            # version CASSEE
code de sortie: 1 (1 = la pipeline echouerait)
[deploy] Version demandée : 1.0.2-bbbbbbb (version actuelle : 1.0.1-aaaaaaa)
[deploy] Base sauvegardée : backups/avant-1.0.2-bbbbbbb-20261109-145943.sql.gz
[deploy] Démarrage de 1.0.2-bbbbbbb (migrations Prisma, puis API)...
[deploy] ÉCHEC : 1.0.2-bbbbbbb n'est pas devenue saine dans les 45 s. Derniers logs de l'API :
[deploy] Retour arrière vers 1.0.1-aaaaaaa...
[deploy] Retour arrière réussi : 1.0.1-aaaaaaa est de nouveau en service.

$ etat apres le retour arriere
IMAGE_TAG=1.0.1-aaaaaaa
backend: colleenp/souverainrecrut-backend:1.0.1-aaaaaaa (Up 8 seconds (healthy))
postgres: postgres:16-alpine (Up 2 minutes (healthy))
--- releases.log ---
2026-11-09T14:57:08+00:00 1.0.0-387bccc OK
2026-11-09T14:58:58+00:00 1.0.0-387bccc OK
2026-11-09T14:59:41+00:00 1.0.1-aaaaaaa OK
2026-11-09T15:00:42+00:00 1.0.2-bbbbbbb ECHEC
2026-11-09T15:00:50+00:00 1.0.1-aaaaaaa ROLLBACK
--- sauvegardes ---
avant-1.0.1-aaaaaaa-20261109-145922.sql.gz
avant-1.0.2-bbbbbbb-20261109-145943.sql.gz
API: "status":"READY","database":"CONNECTED"
```

Figure 14 : Retour arrière automatique : version cassée détectée en 45 s, version précédente remise en service.

```
Serveur : perte de donnees simulee puis restauration depuis la sauvegarde

# Etat initial
  users=1 candidates=1 jobs=0

$ psql -c 'DELETE FROM candidates'     # simule une perte de donnees
  users=1 candidates=0 jobs=0

$ ./restore-db.sh backups/avant-1.0.2-bbbbbbb-20261109-145943.sql.gz
Restauration de backups/avant-1.0.2-bbbbbbb-20261109-145943.sql.gz : la base actuelle sera REMPLACÉE.
Container souverainrecrut-backend-1 Stopped
Container souverainrecrut-migrate-1 Started
Container souverainrecrut-backend-1 Started
Base restaurée. Vérifier : curl -fsS http://127.0.0.1:${API_PORT:-4500}/v1/ready
# Apres restauration
  users=1 candidates=1 jobs=0

$ ./deploy.sh 1.0.0-387bccc            # retour sur la version REELLEMENT publiee sur Docker Hub
[deploy] Version demandée : 1.0.0-387bccc (version actuelle : 1.0.1-aaaaaaa)
[deploy] Base sauvegardée : backups/avant-1.0.0-387bccc-20261109-150152.sql.gz
[deploy] Démarrage de 1.0.0-387bccc (migrations Prisma, puis API)...
[deploy] Déploiement réussi : 1.0.0-387bccc
versions de test supprimees: 2
IMAGE_TAG=1.0.0-387bccc
API: "status":"READY","database":"CONNECTED"
  users=1 candidates=1 jobs=0
```

Figure 15 : Restauration de la base : perte de données simulée puis restauration depuis la sauvegarde.

Bilan

Toutes les exigences du cahier des charges sont couvertes :

Exigence	Comment je l'ai couverte
Reproductibilité	Outillage en conteneurs, Terraform + Ansible idempotent, déploiement par script et

Exigence	Comment je l'ai couverte
	pipeline.
Qualité logicielle	Tests Jest + Supertest, couverture et Quality Gate SonarQube bloquants.
Sécurité	Secrets dans Jenkins / GitLab uniquement, scan Trivy, image non-root, serveur durci.
Traçabilité	Tag <version>-<sha du commit>, historique releases.log.
Déploiement	Image tirée de Docker Hub, jamais reconstruite ; API + PostgreSQL via Compose.
Continuité	Sauvegarde avant chaque déploiement, rollback automatique, restauration testée.

Limites assumées : Jenkins monte le socket Docker de l'hôte (acceptable en TP local, pas en production), déclenchement par scrutation plutôt que par webhook, et Terraform limité à l'inventaire faute de provider pour mon serveur.

Ce que je retiens : une chaîne CI/CD ne sert pas seulement à déployer vite, mais à déployer en confiance : chaque version est testée, tracée, et on sait revenir en arrière.